

스포너는 세 질문의 답이다 — 어디에, 어떤 자리에, 무엇을

지금 스포너는 폐기하고 새로 만든다. 비교 주제는 **Sector 방식 vs 거리 방식(+체크)**. 두 방식 모두 스포너를 쓰므로, 스포너를 **어떻게 배치·정의하느냐**를 먼저 정한다. 이 문서는 제안된 두 방법의 장단점과 실제로 쓰이는 다른 방식 네 가지를 비교한다.

결정 (2026-09-29) — 방법 1 채택, 나머지는 기록으로 보존
비교 결과는 저작 방식과 무관하고(정책은 위치·반경만 봄), 669개 포인트는 이미 배치돼 있어 1번의 단점(노가다)이 추가로 들지 않는다. ③ 베이크는 에디터 도구 2~3일이 드는데 비교 결과에 영향이 없다. → **데이터 형식(손 배치 + ID 표)은 유지, 런타임 스포너만 새로 짤다**. 1번으로 가도 챙길 것: ① 정책은 MonoBehaviour가 아니라 위치·반경 배열을 받는다(체크 스케일 테스트용 합성 좌표) ② 복제본 자식(17개 × 19) 정리 ③ 로드 시 검증 경고. 이 문서의 2~8절은 나중에 맵을 새로 만들거나 배치 방식을 바꿀 때의 참고로 남긴다.

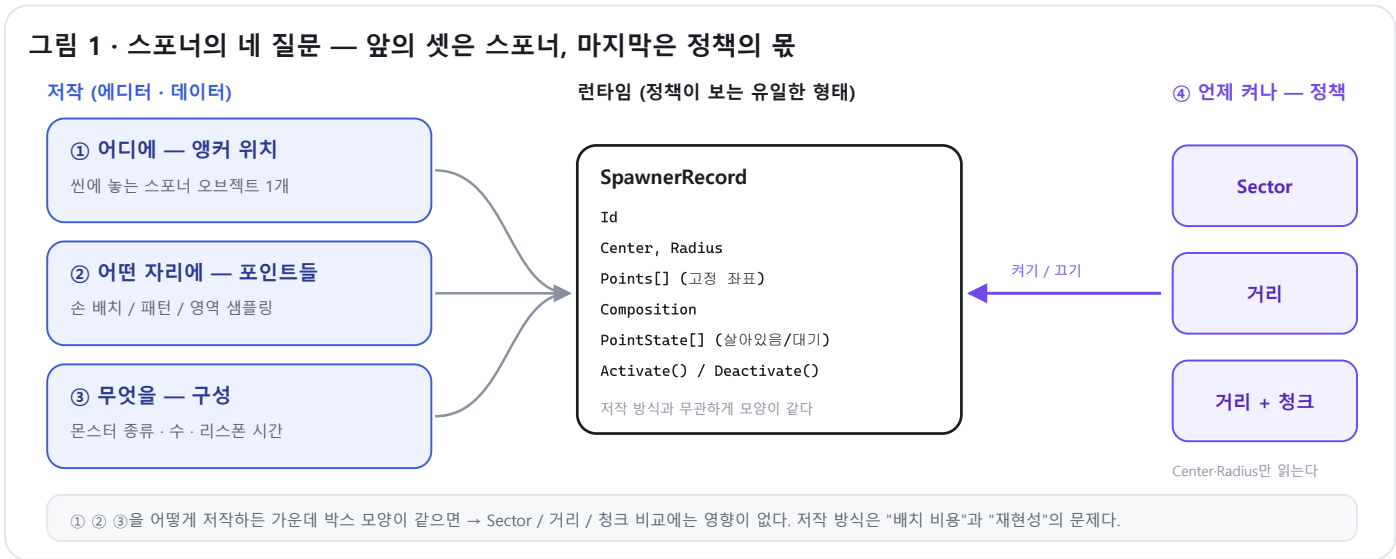
1. 지금 스포너가 보여 주는 문제

BackStreetMap 프리앱과 SpawnerData.json을 직접 세어 본 값 (2026-09-29)

항목	값	읽을 점
스포너 / 자식 포인트	23개 / 669개	포인트 669개를 전부 손으로 배치 — 노가다의 실체
같은 위치에 겹친 자식	19개 스포너 × 17개	자식이 34개인데 17개는 복제본. 데이터는 0~16번만 씀 → 썸과 데이터가 이미 어긋나 있음
썸 ↔ 데이터 연결	spawnerId + pointIndex	자식 순서를 바꾸면 데이터가 다른 자리를 가리킴. 인덱스가 범위 밖이면 경고 없이 <code>continue</code>
스폰 딜레이	363개 전부 5.0초	한 스포너의 17마리가 같은 프레임에 나옴
섹터 소속	계층 (Sector의 자식)	<code>GetComponentInChildren</code> — 정책 데이터가 계층 구조에 박혀 있음
포인트 분포 반경	스포너 중심 ± 5.5m	거리 판정 반경(≈35m)보다 충분히 작음 → 스포너 중심 하나로 판정해도 됨

2. 먼저 나눠 둘 것 — 저작과 런타임

저작 방식은 여러 가지여도, 런타임에 정책이 보는 형태는 하나여야 한다

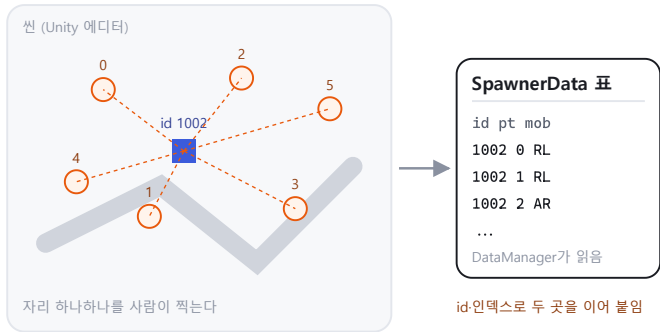


3. 제안된 두 방법

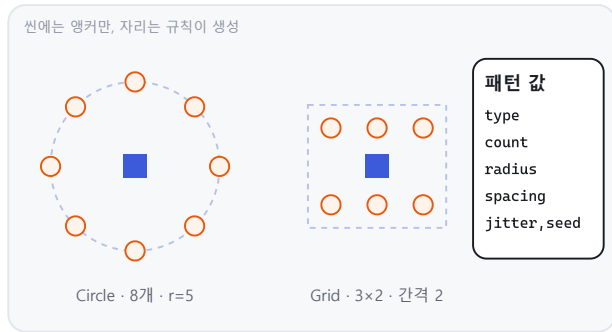
방법 1 = 지금 구조를 정리한 것 · 방법 2 = 포인트를 규칙으로 생성

그림 2 · 방법 1 (손 배치 + ID 표) 과 방법 2 (앵커 + 패턴)

방법 1 · 손 배치 + ID 표



방법 2 · 앵커 + 패턴



둘의 차이는 ② "어떤 자리에"를 사람이 정하느냐, 규칙이 정하느냐. ③ 구성(무엇이 나오나)은 둘 다 표로 둘 수 있다.

방법 1 · 손 배치 + ID 표 (지금 구조를 정리한 형태)

썬에 스포너와 자식 포인트를 배치하고 `spawnerId` 를 붙인다. 어떤 몬스터가 어느 포인트에 나오는지 표(Excel → JSON)에 적고 `DataManager` 가 읽는다.

장점

- 위치를 완전히 통제 — 엄폐물 뒤, 길목처럼 지형 맥락을 살린 배치
- 재현성이 가장 좋다 — 좌표가 고정이라 매번 같은 자리. 측정 비교에 유리
- 구성은 표만 고치면 됨 — 밸런스 조정 때 썬을 열 필요 없음
- 디버깅이 쉬움 — "1002번 3번 포인트"처럼 바로 찾을 수 있음
- 기존 Excel → JSON → DataManager 파이프라인을 그대로 씀

단점

- 배치 노가다 — 지금 669개. 복제본 17개 × 19개가 그 부작용
- 정보가 두 곳에 나뉨 — id 오타, 자식 순서 변경, 인덱스 범위 초과가 조용히 틀어짐
- 썬만 봐서는 무엇이 나오는지 모름 (표를 같이 봐야 함)
- 포인트가 NavMesh 위인지 사람이 확인해야 함
- 합성 스포너 N=10,000 같은 스케일 테스트용 데이터를 만들 수 없음

방법 2 · 앵커 + 패턴

썬에는 앵커만 놓고, 패턴(원형·사각형·격자 등)과 수치만 정한다. 포인트 좌표는 규칙으로 계산하고, 지형 위에 올리려면 `NavMesh.SamplePosition` 으로 투영한다.

장점

- 배치 비용이 스포너당 1번 — 669개가 23개로
- 밀도·수를 숫자 하나로 조정 (count 17 → 12)
- 데이터가 작고 사람이 읽기 쉬움
- 합성 스포너를 코드로 찍어 낼 수 있음 → Phase 7 스케일 테스트와 잘 맞음

단점

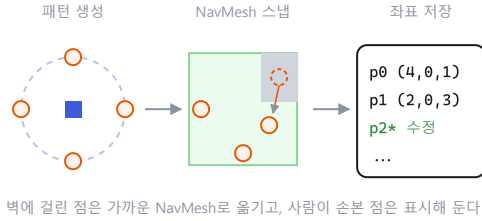
- 지형을 모름 — 벽 속·절벽·NavMesh 밖 포인트. 투영하면 포인트끼리 겹칠 수 있음
- 규칙적인 모양이 인공적 (정확한 원형 진형) → 지터를 넣고 싶어짐
- 지터·투영을 스폰할 때마다 하면 스폰 비용에 계산이 섞이고, 랜덤이면 재현성이 깨짐
- 개별 포인트 예외("이 하나만 옮기고 싶다")를 표현 못 함
- 복도·L자·다각형까지 가면 패턴 종류가 늘어 결국 에디터 도구가 됨

4. 실제로 쓰이는 다른 방식

③~⑤는 ② "어떤 자리에"의 다른 답, ⑥은 ③ "무엇을"의 답이라 어디에나 겹쳐 쓴다

③ 패턴 베이킹 (방법 1 + 2 하이브리드)

자리 생성은 규칙, 저장은 고정 좌표



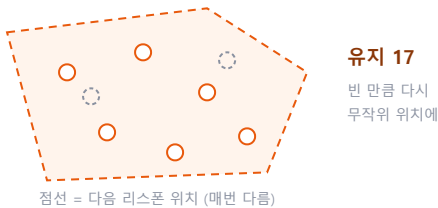
벽에 걸린 점은 가까운 NavMesh로 옮기고, 사람이 손본 점은 표시해 둔다

구현 — 스폰너 인스펙터의 "Bake" 버튼(에디터 스크립트)이 패턴으로 점을 만들고 `NavMesh.SamplePosition` 으로 스냅한 뒤 `Vector3[]` 로 직렬화. 기즈모로 미리보기. 손으로 옮긴 점은 override 표시 → 다시 베이킹해도 보존

- **장점** 저작 비용은 방법 2, 런타임은 방법 1 — **좌표 고정이라 재현성·스폰 비용 순수성 둘 다**
- **장점** 지형 검증을 베이킹 시점에 1번. 예외 포인트도 표현 가능
- **단점** 에디터 도구를 만들어야 함. 지형을 고친 뒤 재베이킹을 잊으면 옛 좌표가 남음 → 검증기 필요
- **변형** 에디터 베이킹 대신 **씬 로드 시 1회 해석**도 가능 (패턴을 표에 두고 로드 때 계산·스냅). 스폰 때 계산만 안 하면 됨

④ 영역 + 수량 (런타임 샘플링)

"이 구역에 RL 17마리 유지"



유지 17
빈 만큼 다시
무작위 위치에

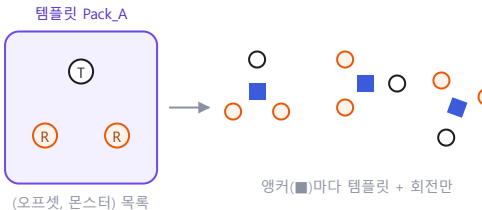
점선 = 다음 리스폰 위치 (매번 다름)

구현 — 볼륨(박스·폴리곤) + 목표 수. 부족하면 영역 안에서 무작위 점을 뽑아 `SamplePosition`, 서로 최소 거리를 두는 Poisson disk 샘플링을 흔히 씀. MMO 필드에서 자주 보는 형태

- **장점** 배치가 가장 적다 (영역 하나). 리스폰 위치가 매번 달라 반복감이 적음
- **단점** 재현성이 가장 나쁘다. 시드를 고정해도 난수 하나를 여러 스폰너가 공유하면, 정책마다 스폰 순서가 달라서 받는 난수도 달라진다 (6절 그림 3)
- **단점** 스폰할 때마다 샘플링·투영 비용 → "스폰 1회 비용" 지표에 섞임
- **비교 트랙 판정** 측정용으로는 부적합. 쓰더라도 결과가 어느 방식에서든 같도록 고정 좌표로 풀어야 함

⑤ 팩(편대) 템플릿

구성 + 상대 배치를 한 묶음으로 재사용



(오프셋, 몬스터) 목록

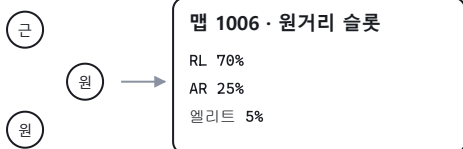
앵커(■)마다 템플릿 + 회전만

구현 — `PackTemplateSO { slots: (offset, monsterId)[ ] }`. 스폰너는 템플릿 참조와 회전만 가짐. 액션 RPG의 "몬스터 팩" 개념과 같은 계열

- **장점** "탱커 1 + 원거리 2" 같은 편대를 맵 전체에서 재사용 — 구성과 배치를 한 번에 수정
- **장점** 좌표가 템플릿에 고정이라 재현성 좋음
- **단점** 오프셋이 지형을 모름 (방법 2와 같은 문제) → 베이킹나 스냅과 같이 써야 함
- **판단** 지금 필드는 RL 360 + AR 3으로 사실상 단일 종이라 템플릿의 이점이 작음. 몬스터 종류가 늘면 가치가 커짐

⑥ 슬롯 + 가중치 표 (구성 레이어)

포인트는 역할만, 실제 몬스터는 표에서 뽑음



맵 1006 · 원거리 슬롯

RL 70%
AR 25%
엘리트 5%

- **장점** 같은 배치로 맵·레벨·지역 변주. 배치와 구성이 완전히 분리됨
- **단점** 가중치 = 랜덤 → 측정할 때는 고정해야 함
- **판단** ①~⑤ 어디에나 겹쳐 쓸 수 있는 층. 이번 비교에는 불필요, 나중에 확장 여지로만

⑦ 밀도 페인팅(지형에 밀도맵을 칠해 샘플링)은 오픈월드용이다. 범위를 일반 필드로 정했으므로 제외한다.

5. 한눈에 비교

● 좋음 · ① 조건부 · ○ 나쁨 — "비교 트랙에 쓸 수 있나"를 기준으로 채점

방식	배치 비용	위치 통제	재현성	지형 안전	스케일 테스트	두 곳 동기화	구현 비용
① 손 배치 + ID 표	○	●	●	①	○	○	●
② 앵커 + 패턴 (런타임)	●	○	①	○	●	①	●
③ 패턴 베이크	●	●	●	●	●	①	①
④ 영역 + 수량	●	○	○	①	●	●	①
⑤ 팩 템플릿	①	①	●	○	●	①	①

재현성 = 같은 입력이면 같은 자리에 같은 몬스터 · 지형 안전 = NavMesh 밖·벽 속 포인트가 안 생김 · 스케일 테스트 = 합성 스폰너 N개를 코드로 만들 수 있음 · 두 곳 동기화 = 씬과 표가 어긋날 위험(●가 위험 낮음). ⑥은 구성 층이라 표에서 뺐다.

③ "무엇을"은 어디에 둘까 — 구성 데이터의 위치

위치	장점	단점
외부 표 (Excel → JSON)	지금 파이프라인 그대로. 기획 값 일괄 수정·밸런싱이 쉬움	id로 씬과 조인 → 어긋나면 런타임에야 발견. 에디터 검증기가 필수
ScriptableObject 참조	id가 없어 조인 오류가 원천 차단. 인스펙터에서 바로 보임	수백 개 값을 한 번에 고치기 불편. Excel 파이프라인과 따로 놓
스포너 컴포넌트에 직접	가장 단순. 한 곳에서 다 보임	재사용 불가. 같은 구성을 바꾸려면 스폰너를 전부 열어야 함

6. 비교 트랙이 스폰너에 요구하는 것

어떤 저작 방식을 고르든 지켜야 Sector / 거리 / 체크 비교가 공정해진다

그림 3 · 난수를 공유하면 정책이 달라질 때 몬스터 자리도 달라진다

공유 난수 (Random 하나)

Sector 정책 A 커짐 → r1, r2 B 커짐 → r3, r4

거리 정책 B 커짐 → r1, r2 A 커짐 → r3, r4

→ 시드가 같아도 A의 자리가 정책마다 달라진다

스포너별 난수 (시드 = id, 포인트, 생애 번호)

Sector 정책 A → a1, a2 B → b1, b2

거리 정책 B → b1, b2 A → a1, a2

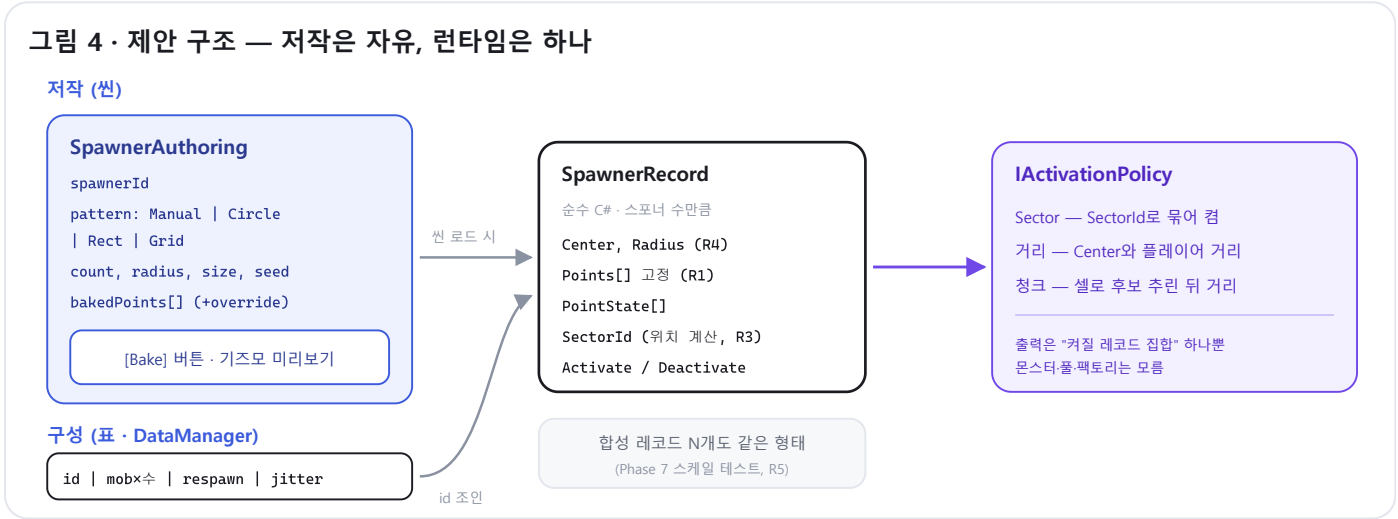
→ 커지는 순서와 무관하게 A는 항상 같은 자리

가장 확실한 해법은 랜덤 자체를 런타임에서 없애는 것 (③ 베이크). 지타·가중치를 꼭 쓰려면 스폰너별 시드로 결정적으로 만든다.

#	요구	이유
R1	스폰 시점에는 좌표 계산 없음	패턴 해석·NavMesh 투영이 스폰마다 돌면 "스폰 1회 비용" 지표가 저작 방식에 오염됨. 베이킹 또는 로드 시 1회
R2	랜덤은 스폰너별 시드로 결정적	그림 3. 공유 난수는 정책마다 뽑는 순서가 달라짐
R3	섹터 소속은 계층이 아니라 위치로 계산	스폰너를 평평하게 두고 "어느 섹터 볼륨에 중심이 들어가나"로 로드 시 계산 → 세 정책이 같은 스폰너 집합 을 공유. 섹터 경계를 옮겨도 재배포 불필요
R4	스폰너 반경에 상한 (예: $\leq 8m$)	정책은 Center-Radius만 봄. 스폰너가 너무 크면 "반만 반경 안" 상태가 생김. <code>CullingGroup</code> 은 <code>BoundingBox(center, radius)</code> 를 받으므로 Radius를 레코드에 저장
R5	코드로 합성 스폰너를 만들 수 있음	Phase 7 스케일 테스트 (N = 23 ~ 10,000). 손 배치만 되는 구조면 불가능
R6	에디터 검증기	id 존재 · 구성 수 $\leq$ 포인트 수 · 포인트가 NavMesh 위 · 중복 좌표 없음(지금의 복제본 17개를 잡아냄) · 반경 상한

7. 제안 — 1과 2는 경쟁이 아니라 "포인트 생성기"의 두 종류

패턴 종류에 Manual(손 배치)을 넣으면 방법 1이 방법 2의 특수한 경우가 된다



추천 — ③ 패턴 베이킹 + 구성은 표

Manual / Circle / Rect / Grid 네 가지 패턴만 두고, 결과 좌표를 베이킹해서 저장한다. 엄폐물 뒤처럼 자리를 꼭 지정해야 하는 곳은 Manual(= 방법 1), 넓은 공터는 Circle/Grid(= 방법 2). 구성(몬스터 × 수 · 리스폰)은 지금처럼 Excel → JSON 표에 두되, **포인트 인덱스 대신 "몬스터 × 수"**로 적어 포인트 순서가 바뀌어도 데이터가 깨지지 않게 한다. 조인 오류는 R6 검증기가 에디터에서 잡는다.

이 추천이 푸는 것

노가다 (669 → 23 + 필요한 곳만 Manual) · 재현성 (좌표 고정) · 스폰 비용 순수성 (계산 없음) · 지형 안전 (베이킹 시 스냅) · 스케일 테스트 (레코드를 코드로 생성) · 섹터 계층 의존 제거

대가

에디터 도구(베이킹 버튼 · 기즈모 · 검증기)를 만드는 비용. 지형을 고친 뒤 재베이킹을 잇는 위험 → 검증기가 "NavMesh 밖 포인트"로 잡음. 에디터 도구가 부담이면 변형안(로드 시 1회 해석)으로 시작해도 R1은 지켜진다.

구현 스케치

```
// 저작 - 씬에 놓는 컴포넌트. 런타임에는 레코드로 변환되고 끝.
public class SpawnerAuthoring : MonoBehaviour
{
    public int spawnerId; // 구성 표와 조인
    public SpawnPattern pattern; // Manual | Circle | Rect | Grid
    public int count; public float radius; public Vector2 size; public int seed;
    [SerializeField] Vector3[] _bakedPoints; // [Bake] 결과 (Manual이면 자식에서 수집)
    [SerializeField] bool[] _overridden; // 손으로 옮긴 점 - 재베이크해도 보존
}

// 런타임 - 정책이 보는 유일한 형태. MonoBehaviour 아님 → 합성 N개를 코드로 생성 가능
public sealed class SpawnerRecord
{
    public readonly int Id, SectorId; // SectorId는 로드 시 위치로 계산
    public readonly Vector3 Center; public readonly float Radius;
    public readonly Vector3[] Points; // 고정 좌표
    PointState[] _state; // 살아있음(인스턴스) / 재등장 예정(절대 시각) / 비어있음
    public bool IsActive { get; private set; }
    public void Activate(); public void Deactivate();
}
```

8. 정할 것

#	질문	후보	이 문서의 제안
1	포인트 좌표를 누가 확정하나	손 배치 / 런타임 패턴 / 에디터 베이크 / 로드 시 해석	에디터 베이크 (부담되면 로드 시 해석)
2	구성 데이터 위치	Excel 표 / ScriptableObject / 컴포넌트	Excel 표 + 에디터 검증기
3	구성을 적는 단위	포인트별 지정 / (몬스터, 수) 목록	(몬스터, 수) — 포인트 순서와 분리
4	패턴 최소 집합	Manual · Circle · Rect · Grid (+ 지터)	넷으로 시작, 지터는 스포너별 시드로
5	섹터 소속	계층 / 위치 계산 / 표에 명시	위치 계산 (중심점 기준)
6	스포너 반경 상한	없음 / 8m / 10m	지금 분포(5.5m) 근처로 상한